

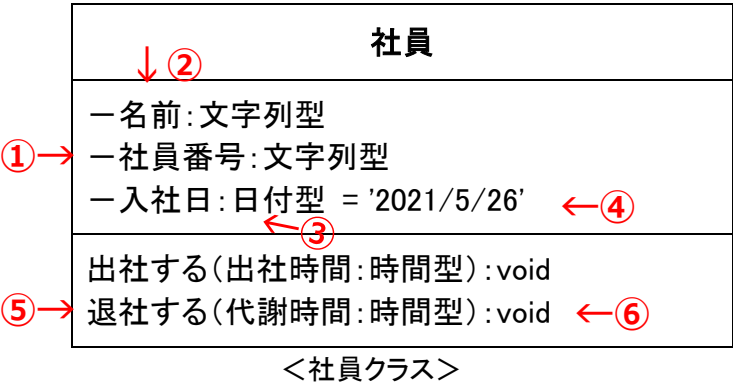
2年生 SE コース 開発基礎

～クラス図について～

クラス図とは

クラスの構造・関係性を表現する図

クラス図の描き方(基本)



1. 可視性

①の箇所、この例では「-」となっている箇所にあたりますが、こちらは可視性を表しています。可視性とは、この後に出てくる「属性」や「操作」に対してアクセスが可能となる範囲を表現するものです。可視性には以下の4種類があります。

記 号	意 味
+	すべてのクラスからアクセスが可能
-	自クラスからのみアクセスが可能
#	自クラスおよび継承されているクラスからのみアクセスが可能
~	同一パッケージ内のクラスからのみアクセスが可能

2. 属性名

②の箇所は「属性名」です。属性は、そのクラスが持つ要素を意味しています。今回の「社員」というクラスには、「社員名(名前)」「社員番号」、「入社日」といった社員に関する属性が入ります。

3. 型

③の箇所には、「型」が入ります。型といってもクラスで言う型ではなく、その属性が文字列で表されるか、数値で表すかといった、属性を表現する型が入ります。プログラミングにおいては、int か string かといった具合ですね。

③の属性は社員番号となるので、文字列型ではなく数値型などにしても問題はありません。

4. 初期値

④の箇所には、「初期値」が入っています。こちらは読んで字の通り、最初にあらかじめ入れておく値(デフォルト値)になります。以上の 1～3 を 1 行で、

「属性名:型=初期値」というように記述します。

5. 操作名

続いて、クラスの下半分の領域に入ります。⑤では、そのクラスで行う事のできる「操作」を表現します。プログラミングで言う、メソッドや関数のこと。

「操作名(引数:引数の型)」と表します。

6. 戻り値

最後に⑥は操作の「戻り値」です。戻り値を返さない場合は void とします。操作名と合わせて、

「操作名():戻り値」と表します。

クラス図の詳細な表現について

1. 関連

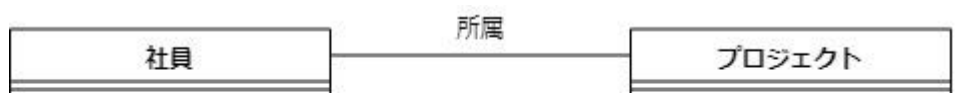
クラスが複数あり、それらに何かしらの関係があるときは、「関連」として表現します。

例えば、

A のクラスが B のクラスの操作を呼び出す関連（A のフィールドに B のインスタスを入れてメソッドを利用する）

A のクラスが B のクラスのデータを属性として保持している関連

などがあります。関連を表現するには、A と B のクラスを線で結び、そこに関連名を付与します。



※クラスの中身は省略しています

上記の例では、「所属」が関連名となります。

2. 多重度

関連のあるクラス間において、クラス A から見たクラス B の存在しうる数を表したものが「多重度」です。

例としては、以下のようになります。



※クラスの中身は省略しています

この図から分かるのは、

1 つの部署から見て、社員は 1 人以上いること

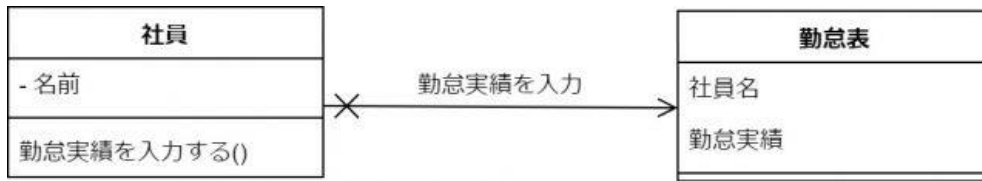
1 人の社員から見て、所属する部署は 1 つである事

が分かります。多重度の数字の意味は以下のようになります。

表 記	意 味
n	値の通り
"0..n" または "*"	0 以上
"1..*"	1 以上
"m..n"	m から n まで

3. 誘導可能性

関連のあるクラス同士のやりとりが、単方向からのみである場合、その方向性を明確に表す事ができ、「誘導可能性」として表現する事ができます。実際の例で見てみましょう。



このように、矢印とバツで表現する事が可能です。

4. 依存

「依存」は、クラス間で関係性はあるが、あまり強くない関係性の場合に使います。実際の例で見てみると分かりやすいかと思います。



※クラスの中身は省略しています

ここで依存関係となっているのは、「会社」と「社員」クラスです。会社は「雇用する」操作によって社員クラスを生成しますが、その後操作を行わない一時的な関係性です（引数で受け取ったインスタンスを利用したり、戻り値でインスタンスを返すなど）。このような強くない関係性の場合、依存を活用します。

5. 集約

関連のあるクラス同士が、「全体」と「部分」の関係にあるとき、「集約」として表現します。具体的な例を挙げると、「部署」と「社員」といった関係が「全体」と「部分」にあたります。実際に集約を図で表すと、以下ようになります。



※クラスの中身は省略しています

6. コンポジット

コンポジットは、集約の一種になります。コンポジットは「全体」インスタンスが「部分」インスタンスの生成や削除を担っている場合に使用されます。

また、全体インスタンスが削除されると、基本的には、連鎖して部分インスタンスも削除される関係になります。UML でコンポジットは菱形部分が黒くなります。

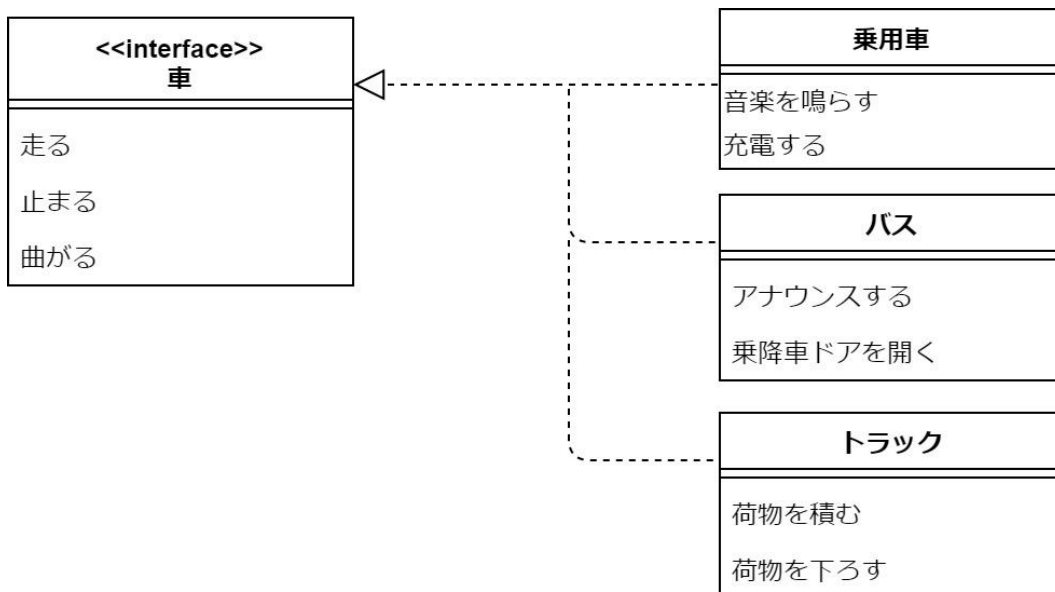


※クラスの中身は省略しています

7. 汎化（インタフェース）

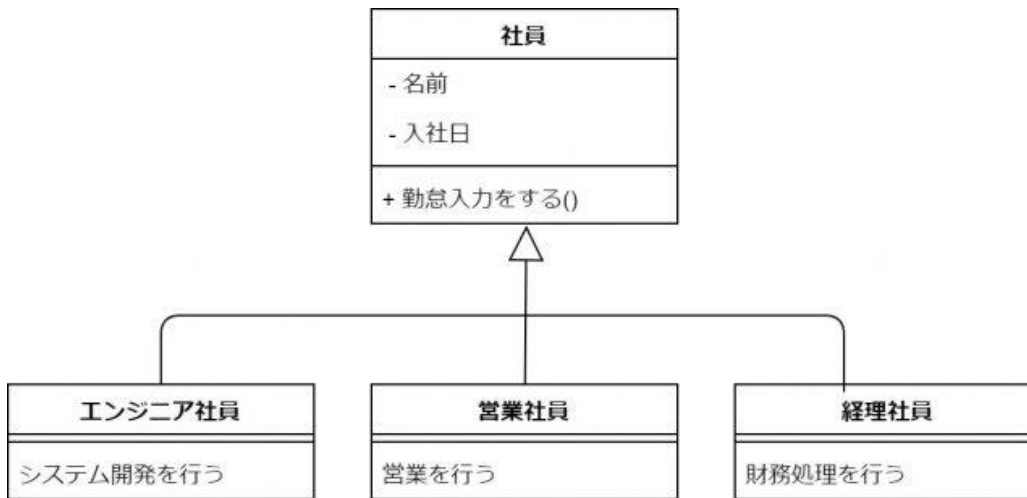
「インタフェース」は、クラスが実現するうえで必要な共通操作のみを定義したクラスを言います。ルールとしてインタフェースには操作の具体的な処理は記載せず、具体的な処理は「サブクラス」で定義します。

このインタフェースとサブクラスのことを「実現」と言います。なお、実現は汎化の一種です。例として、車をインタフェースとしてみると、「乗用車」、「バス」、「トラック」などがサブクラスになります。UML でこれを表現すると、以下ようになります。



8. 汎化（継承）

「汎化」はいわゆる継承です。クラス A の型を継承し、具体化したクラス B があるとき、A と B の関係性は「汎化」と言えます。このとき、クラス A を「スーパークラス」と言い、クラス B を「サブクラス」と言います。UML で描画すると、以下のようになります。



9. 矢印のまとめ

矢印の種類	意 味
	関連 (フィールドに別のクラスのインスタンスを保持し利用する場合。)
	誘導可能性 (関連がある場合で向きがあるときは矢印と×をつける。向きが未定の時は×をつけなくてもよい。)
	依存 (関連するが結びつきが弱いもの。例えばメソッドの引数や戻り値で受け渡しして利用するようなもの。)
	集約
	コンポジット
	実現 (インターフェースの実装)
	汎化 (継承関係)